PROJECT-BASED LEARNING
PHASE-I EVALUATION
Virtual File System

Team ID: DSCPP-III-2026-T123

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information
Team ID: DSCPP-III-2026-T123
Semester: 3rd
Section: G,H,ML2,DS2
Domain: Data Structures / OOP

Team Members

1. Tushar Singh – 2510011704
2. Tanish Naithani – 25100011477
3. Rounak Tayal – 2510370114
4. Aditya Rawal – 2510380204

Mentor: Mr. Akshay Rajput

Interactions completed: 3

Problem Statement & Motivation

Problem Statement

File systems manage storage, directories, and metadata invisibly — internal workings (block allocation, caching, recovery) are hidden from users, making it hard to learn how they actually work.

Motivation

- OS hides file-system internals
- Hard to learn practically
- Existing tools are black-box

Target Users

- CS students learning OS/DS
- Educators (teaching aid)
- Future: embedded/small systems

Objectives

1. Design and build a working Virtual File System in C++
2. Implement core file/directory operations (CRUD)
3. Integrate DS (trees, hashing, bitmaps) + OOP (classes, inheritance)
4. Test via CLI-driven operations and edge cases
5. Demonstrate applicability to OS/DS learning & future scalability

Key Objective: Deliver a functional CLI-based VFS supporting file/directory ops, caching, and journaling.

Proposed Solution

Proposed Approach

1. CLI command input
2. Parse & route to file/dir manager
3. Tree + hash table lookup
4. Block manager + bitmap allocation
5. Output result / status

Key Features

- Fixed-size block virtual file
- Tree-based directory hierarchy
- LRU cache for blocks
- Journaling & snapshots
- Filesystem consistency check

Integration of PBL Course Concepts

Course	Concepts Applied	Module
DS in C	Trees, Hashing, Bitmaps, Linked Lists	Core FS Engine
OOP in C++	Classes, Inheritance, Polymorphism	System Design

Directory tree → hierarchy; hash table → fast file lookup; bitmap → free-block tracking; classes → modular disk/file/cache/journal components.

System Architecture / Workflow

CLI Input → Command Parser → File/Directory Manager → Block Manager (Bitmap) → Virtual File → Output

Major Components: Virtual File, Block Manager, Directory Tree, Cache, Journal, CLI

Data Flow: User command → parsed → resolved via tree/hash → block read/write → result shown

Technology Stack & Methodology

Technology Stack

- Language: C++
- Storage: In-memory / flat binary file
- IDE: VS Code
- Version Control: Git/GitHub

Methodology

1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Evaluation

Initial Progress & Team Contribution

Progress Achieved

- Literature study on file systems completed
- Requirements identified
- System architecture designed
- Initial disk/block module implemented

Member	Role	Contribution
Aditya Rawal	Lead	Architecture, Block Manager
Tanish Naithani	Developer	Directory Tree
Tushar Singh	DB/Testing	Testing, Metadata
Rounak Tayal	Developer/Docs	CLI, Documentation

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal & Design
2. Phase-II: Development
3. Phase-III: Final Implementation

Expected Outcomes

- Working CLI-based VFS
- File/dir ops, caching, journaling
- Scalable, extensible design

References

1. Balagurusamy, *OOP with C++*
2. Kanetkar, *Data Structures Through C*
3. GeeksforGeeks – DSA
4. Programiz – C++ Programming

Thank You

Questions & Suggestions